Fashion Agent: An LLM-Powered Multimodal Fashion Retrieval System

Based on Retrieval-Augmented Generation
and Direct Routing Agent Architecture

LLM Thesis — Technical Report

University of Information Technology (UIT)
Faculty of Computer Science

Author: [Student Name]
ID: [Student ID]
Advisor: [Advisor Name]
Semester: [Semester / Academic Year]
Date: March 2026

fashion_agent_article.tex · pdfLaTeX

Abstract

We present **Fashion Agent**, an intelligent multimodal fashion retrieval system that combines Retrieval-Augmented Generation (RAG) with a deterministic *Direct Routing* orchestration layer. The system evolves through two phases: *RAG v1.0*, a fixed nine-step hybrid-search pipeline using BM25 and Marqo-FashionSigLIP (768-d) vector embeddings; and *Agent v2.0*, which introduces intent classification, a template-based slot gate, session memory backed by PostgreSQL, and a direct Gemini 2.5 Flash orchestrator requiring only 1–2 LLM calls per conversational turn — with no iterative ReAct loop. A BAAI/BGE cross-encoder reranker further improves retrieval precision. End-to-end evaluation targets show Hit Rate@5 $\geq$ 93%, MRR $\geq$ 0.80, and a P95 latency under 15 seconds. The Clothie Web frontend (Flutter, served via Nginx) connects to the FastAPI backend over a Cloudflare Tunnel. This article details the technical architecture, processing workflows, and key design decisions behind the system.

Contents

1 Introduction

Fashion e-commerce demands more than simple keyword matching. Shoppers express intent in colloquial language ("something smart for a Monday office meeting"), mix languages, and expect coherent multi-turn conversations. Classical retrieval systems fail on all three counts.

Fashion Agent addresses these limitations through three complementary mechanisms:

(1) **Hybrid retrieval** — lexical BM25 for exact color/category matching combined with a fashion-specific SigLIP encoder for semantic similarity.

(2) **LLM-orchestrated reasoning** — Gemini 2.5 Flash classifies intent and extracts structured slots in a single call, then routes deterministically through a template-based slot gate and hybrid search, and synthesises a natural-language response — with no iterative ReAct loop.

(3) **Persistent session memory** — PostgreSQL stores cross-turn context, liked products, and accumulated user preferences.

The system processes **text-to-image** search (PATH 1): a natural-language query returns ranked product images with styling commentary, backed by a dataset of clothing images enriched with LLM-generated captions and fine-grained color labels.

1.1 Scope of This Article

This article focuses exclusively on the *technical architecture and processing workflows* of Fashion Agent v2.0 (PATH 1). Section 2 covers data preprocessing; Section 3 describes the hybrid retrieval core (RAG v1.0); Section 4 details the full agent pipeline; Section 5 presents evaluation metrics; Section 6 outlines the deployment stack.

2 Data Preprocessing Pipeline

2.1 Dataset

Training data originates from the *Clothing Dataset Full* (Kaggle), a curated set of fashion product images. Each sample provides: an image identifier, a category label, and an optional color annotation. Two preprocessing problems motivate the pipeline: (1) noisy labels (`Not sure`, `Skip`, `Other`) that degrade retrieval precision; (2) short labels that are too sparse for meaningful semantic embedding.

2.2 Preprocessing Workflow

Figure 1 shows the full pipeline.

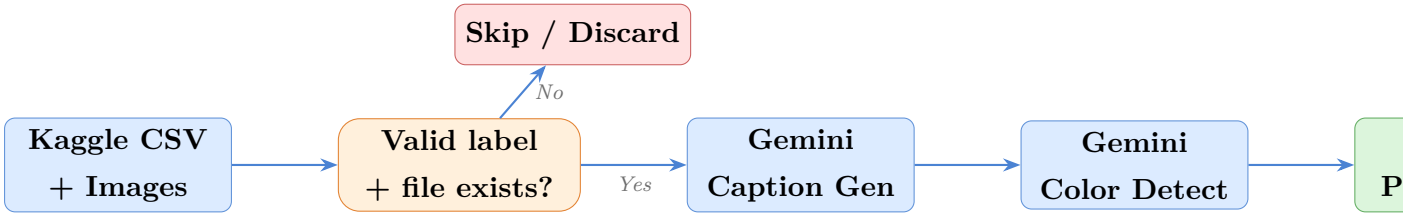


Figure 1: Data preprocessing pipeline: noisy records are discarded, valid items receive LLM-generated captions and color labels, then are stored in PostgreSQL.

Label filtering. Records whose `label` field belongs to the exclusion set {`"Not sure"`, `"Skip"`, `"Other"` or whose image file is missing, are silently skipped.

Caption generation. Each product image is passed to *Gemini 2.5 Flash* alongside a structured prompt that requests a 30 – 40-word description emphasising fabric, silhouette, construction detail, and aesthetic. Colors and brand names are deliberately excluded to keep the representation general. The resulting *caption* acts as a semantic token — richer than a bare category label and therefore much more informative for embedding-based retrieval.

Color labeling. A second Gemini call extracts a precise, granular color annotation (e.g., *"Charcoal Grey"* rather than *"Grey"*). This field feeds BM25 keyword matching and soft-filter logic.

Batch processing & checkpointing. Items are processed in batches of 20 with `asyncio.sleep()` throttling. PostgreSQL is committed after every batch, providing crash resilience across the full dataset ingestion run.

2.3 Schema Design

Column	Type	Description
`image_id`	TEXT	Unique slug from Kaggle filename
`label`	TEXT	Fashion category (T-Shirt, Dress, ...)
`color`	TEXT	Fine-grained color from Gemini
`caption`	TEXT	3040-word LLM-generated description
`image_path`	TEXT	Absolute path to JPEG on disk
`created_at`	TIMESTAMPTZ	Ingestion timestamp

Table 1: PostgreSQL `fashion_items` schema.

3 Hybrid Retrieval Core (RAG v1.0)

3.1 Overview

The retrieval backend combines two complementary search paradigms to maximize both *precision* (exact keyword matching) and *recall* (semantic similarity):

- **BM25 (lexical)**: Indexes the concatenated string `"⟨label⟩ ⟨color⟩"` for each item. Excels at exact category/color queries ("navy blue shirt").
- **Marqo-FashionSigLIP (semantic)**: Encodes text queries and images into shared 768-dimensional vectors. Captures synonyms, style descriptions, and nuanced semantics ("relaxed-fit woven cotton pullover").

Results from both retrievers are merged with *Reciprocal Rank Fusion* (RRF), then re-ranked by a cross-encoder reranker.

3.2 Marqo-FashionSigLIP Embedding Model

Why FashionSigLIP over FashionCLIP? The `Marqo/marqo-fashionSigLIP` model produces 768-d vectors (vs. 512-d for FashionCLIP), is fine-tuned on fashion-specific data, and uses the SigLIP loss (sigmoid-based pairwise rather than softmax), which generalises better on small downstream domains. It is served through the `open_clip` library, which offers native Apple MPS (Metal Performance Shaders) support — important for development on Apple Silicon hardware.

L_2-normalised embeddings ensure that dot-product similarity equals cosine similarity:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}, \qquad \text{sim}(\hat{\mathbf{u}}, \hat{\mathbf{v}}) = \hat{\mathbf{u}} \cdot \hat{\mathbf{v}} \in [-1, 1].$$

All item captions and labels are embedded offline during indexing and stored in a **Qdrant** vector database. At query time, the user's text is encoded and the k-nearest neighbors are retrieved by approximate nearest-neighbor search.

3.3 Reciprocal Rank Fusion (RRF)

RRF avoids the incompatibility of raw BM25 and cosine scores by blending *rank positions* rather than absolute values. The agent employs a **three-way** fusion over BM25, image-vector, and text-vector retrievers:

$$\text{score}(d) = \frac{w_{\text{bm25}}}{k + r_{\text{bm25}}(d) + 1} + \frac{w_{\text{img}}}{k + r_{\text{img}}(d) + 1} + \frac{w_{\text{text}}}{k + r_{\text{text}}(d) + 1}$$

where $k = 60$ is a rank smoothing constant, $w_{\text{bm25}} = 2.5$, $w_{\text{text}} = 1.5$, and $w_{\text{img}} = 1.0$. The higher weight on BM25 reflects the value of exact keyword matches for category and color queries; text-vector search provides semantic coverage; image-vector search captures visual style.

3.4 Query Expansion

The pipeline supports *optional* Gemini-driven query expansion (synonym sub-queries), but it is **disabled** in the agent orchestrator by default (`use_query_expansion=False`) to reduce latency. Expansion may be enabled for RAG v1.0 standalone benchmarks.

3.5 BGE Cross-Encoder Reranker

After fusion, the top-N candidates are passed to **BAAI/bge-reranker-v2-m3**, a *cross-encoder* model that jointly encodes the query and each candidate passage and produces a fine-grained relevance score. Unlike bi-encoders (which embed query and document independently), the cross-encoder has access to query-document interactions, yielding significantly higher precision at the cost of higher latency. The reranker is applied to the top-20 RRF results, returning the top-6. A score threshold of ≥ 0.25 is applied after reranking; results below this floor are dropped (minimum one result is always returned to avoid empty responses).

3.6 Retrieval Pipeline Architecture

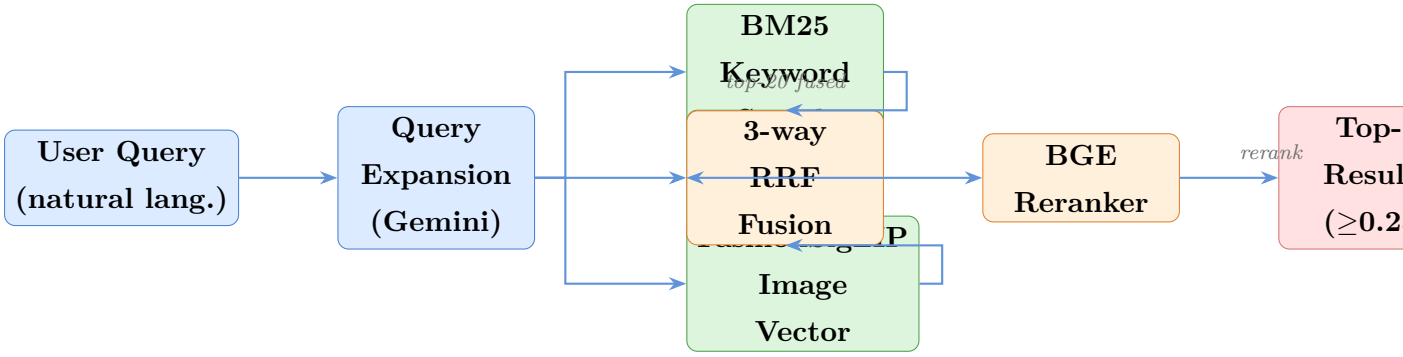


Figure 2: Hybrid retrieval pipeline. The user query is searched in parallel via BM25, text-vector search, and FashionSigLIP image-vector search (query expansion is disabled in the agent by default). 3-way RRF fuses the ranked lists with weights $w_{\text{bm25}} = 2.5$, $w_{\text{text}} = 1.5$, $w_{\text{img}} = 1.0$; a BGE cross-encoder reranker selects the final top-6 (score threshold ≥ 0.25).

4 Agent v2.0: Direct Routing Pipeline

4.1 Architectural Philosophy

RAG v1.0 executes a fixed nine-step pipeline regardless of query complexity or available context. Agent v2.0 replaces this with a *deterministic linear pipeline*: **Classify → Slot Gate → Search → Synthesise**. Gemini 2.5 Flash is called exactly twice per turn in the happy path (once for intent + slot extraction, once for response synthesis); there is no iterative loop, no dynamic tool selection, and no runaway latency risk.

The design follows four principles:

(i) **Intent-first** — classify before acting, so downstream modules receive structured, validated signals.

(ii) **Slot gate, not LLM clarification** — missing search parameters trigger pre-written template questions (zero extra LLM calls), with a maximum of three clarification turns.

(iii) **Deterministic routing** — intent maps directly to a code path; no Gemini reasoning is needed to decide which tool to call next.

(iv) **Persistent context** — session preferences accumulated in PostgreSQL influence each new search within the conversation.

4.2 End-to-End Agent Workflow

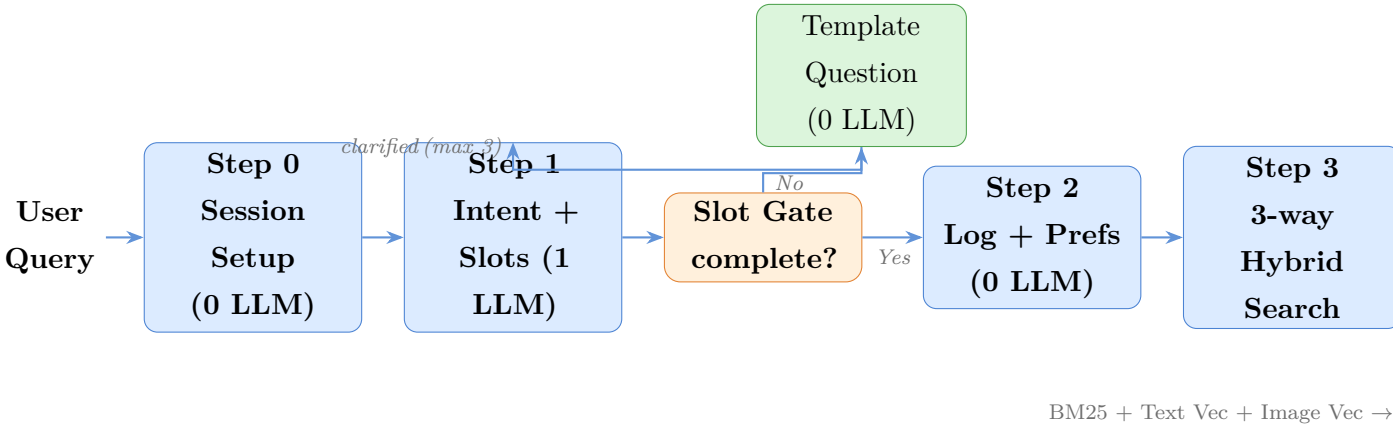


Figure 3: End-to-end Agent v2.0 workflow (Direct Routing Pipeline, PATH 1). A template-based slot gate handles ambiguous queries with zero additional LLM calls. The happy path costs exactly 2 LLM calls (intent + synthesis).

4.3 Step 1: Intent Classification

A few-shot prompt sent to Gemini maps each incoming message to one of seven intent classes (Table 2). The model simultaneously extracts six structured *slots* ({`category`, `color`, `fabric`, `fit`
and a *refined query* string — all in a single LLM call.

Intent	Confidence trigger	Agent action
`text_search`	≥ 0.6	Slot gate → hybrid search → synthesis
`outfit_request`	≥ 0.6	Search + generate outfit advice
`follow_up`	any	Merge slots from cache, refine search
`product_select`	any	Match number(s) against cached results, confirm save
`view_selections`	any	List saved items from PostgreSQL
`out_of_scope`	any	Template refusal (0 LLM)
`unclear`	< 0.6	Slot gate / clarification (up to 3 turns)

Table 2: Intent taxonomy and corresponding agent actions.

History-aware classification. The last four conversation turns are prepended to the prompt. This allows the classifier to correctly label *"the white one"* as a `follow_up` rather than `unclear` when a prior turn established the category context.

4.4 Step 2: Slot Gate

The slot gate is a *zero-LLM* completeness checker that runs before any search is executed. For `text_search` intents, it merges newly extracted slots with those accumulated across the session (TTL cache, 30 minutes). A search proceeds only when the *minimum slot set* is satisfied:

`category` **AND** `color` **AND** (`fabric` **OR** `fit`)

If any required slot is missing and fewer than three clarification turns have occurred, the gate emits a *template question* (no Gemini call) targeting the most critical missing slot. After three turns the gate releases the search with whatever slots are available, preventing infinite loops.

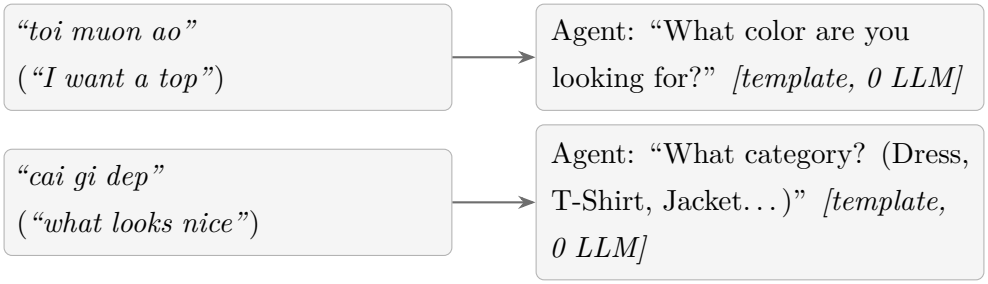


Figure 4: Slot gate template clarification examples. Vague queries trigger pre-written questions targeting the most critical missing slot — no additional LLM call needed.

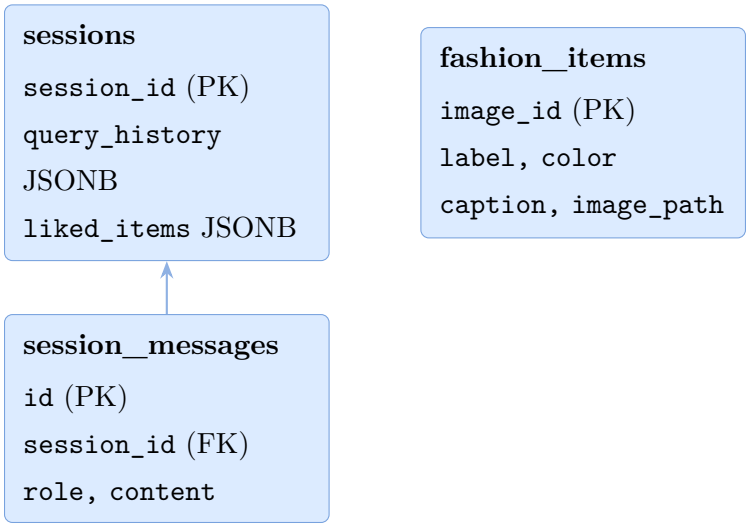
`follow_up` handling. For `follow_up` intents the gate skips completeness checking and simply merges the new slots with the cached ones, then composes a refined query from the accumulated slot values using `compose_refined_query_from_slots()`.

4.5 Step 3: Memory Agent (PostgreSQL)

Before executing any retrieval, the agent loads accumulated session preferences from PostgreSQL.

- **Query history** — JSONB array of past queries with extracted filters (`color`, `category`).
- **Liked items** — JSONB array of products the user explicitly approved.
- **Derived preferences** — the agent counts most-frequent colors and categories across history to construct a soft preference bias that enriches the search query.

Schema highlights. Three tables support memory:



4.6 Step 4: Hybrid Search Execution

Once the slot gate releases, the agent calls `_route_and_execute()`, which invokes the full hybrid retrieval pipeline deterministically — no LLM reasoning required.

(1) **BM25** retrieves top-5 candidates by keyword match on `label + color`.

(2) **Image Vector ANN** retrieves top-5 by FashionSigLIP cosine similarity (768 d).

(3) **Text Vector ANN** retrieves top-5 by text-vector cosine similarity.

(4) **3-way RRF** fuses the three ranked lists with $w_{\text{bm25}} = 2.5$, $w_{\text{text}} = 1.5$, $w_{\text{img}} = 1.0$.

(5) **Soft Relevance Filter** (RapidFuzz) applies category/color fuzzy matching on the fused list.

(6) **BGE Reranker** re-scores the top-20 RRF candidates; results below 0.25 are dropped.

(7) The final **top-6** results are returned to the synthesis step.

4.7 Step 5: LLM Response Synthesis

The top-6 retrieved products, the full conversation history (last 6 turns), and user preferences are streamed to a final Gemini 2.5 Flash call that:

(1) selects the most relevant items to mention;

(2) generates a concise response *in the user's language* (bilingual: English or Vietnamese);

(3) appends a `styling_suggestion` for `outfit_request` queries;

(4) logs token usage (input + output + thinking tokens) to the `llm_token_usage` table in PostgreSQL.

5 Evaluation

5.1 Metrics

Metric	Definition	RAG v1.0	Agent v2.0
Hit Rate@5	≥ 1 correct item in top-5	$\geq 90\%$	$\geq 93\%$
Recall@5	Fraction of relevant items recalled	$\geq 85\%$	$\geq 88\%$
MRR	Mean Reciprocal Rank of first correct item	≥ 0.75	≥ 0.80
Faithfulness	RAGAS — no hallucinated product info	—	≥ 0.87
Task Compl.	Query handled without crash	—	$\geq 95\%$
P95 Latency	End-to-end wall time (95th percentile)	$< 15\,\text{s}$	$< 15\,\text{s}$

Table 3: Target evaluation metrics comparing RAG v1.0 and Agent v2.0.

5.2 Comparative Analysis

Criterion	RAG v1.0	Agent v2.0
Query handling	Fixed 9-step pipeline	Direct Routing Pipeline
Ambiguous queries	Search with partial info	Proactive clarification
Multi-turn context	No memory	PostgreSQL session memory
Embedding model	FashionCLIP (512-d)	FashionSigLIP (768-d)
Query expansion	None	Gemini synonym expansion
Reranker	None	BGE cross-encoder
Response generation	Product list only	Natural language + styling
Storage	Local CSV files	PostgreSQL (server-grade)
Out-of-scope detection	None	LLM intent gate

Table 4: Feature comparison: RAG v1.0 vs. Agent v2.0.

5.3 Token Usage Measurement

Accurate token tracking is essential for reporting API cost and understanding the computational budget of each conversation turn. The system implements *end-to-end token*

instrumentation covering every Gemini call in the pipeline.

Instrumentation architecture. Token counts are extracted directly from the Gemini SDK's `response.usage_metadata` object, which exposes three fields:

- **`prompt_token_count`** — tokens consumed by the input prompt (system message + few-shot examples + user context).
- **`candidates_token_count`** — tokens in the model's generated output.
- **`thoughts_token_count`** (Gemini 2.5 Flash) — hidden reasoning tokens used internally during extended thinking; currently *not persisted* but available for future capture.

Each call is tagged with a `call_name` string (`"intent"` or `"synthesis"`) and written asynchronously to PostgreSQL via `log_token_usage()`. A database view (`session_token_summary`) aggregates totals per session on-the-fly:

```
CREATE OR REPLACE VIEW session_token_summary AS
SELECT
  s.session_id, s.user_name,
  MAX(u.model_name)                       AS model_name,
  SUM(u.input_tokens)                     AS total_input_tokens,
  SUM(u.output_tokens)                    AS total_output_tokens,
  SUM(u.input_tokens + u.output_tokens)   AS total_tokens,
  s.created_at::date                      AS session_date
FROM llm_token_usage u
JOIN user_sessions s USING (session_id)
GROUP BY s.session_id, s.user_name, s.created_at;
```

An analytics endpoint (`GET /analytics/token-usage`) exposes these aggregates to the evaluation dashboard.

LLM calls per conversation turn. Each user message triggers exactly **two** Gemini API calls in the happy path, as shown in Table 5.

Call	Tag	Prompt contents	Input est.	Output est.
Intent classifier	`intent`	System instruction + 5-shot examples + recent history + user query	350–600	50–120
Response synthesis	`synthesis`	Retrieved products (up to 6) + history (last 6 turns) + preferences + user query	1 500–3 000	200–500
Total per turn			1 850–3 600	250–620

Table 5: Estimated token budget per conversation turn (Gemini 2.5 Flash, no query expansion, top-6 results). The intent call is small and fast; synthesis dominates input cost due to the product context window.

Cost estimation. Using the Gemini 2.5 Flash pricing schedule (input \$0.075 / 1M tokens; output \$0.30 / 1M tokens), the per-turn API cost is approximately:

$$C_{\text{turn}} = \frac{T_{\text{in}}}{10^6} \times 0.075 + \frac{T_{\text{out}}}{10^6} \times 0.30 \approx \$0.0002 - \$0.0005$$

At this rate, **1 000 conversation turns** cost roughly \$0.20–\$0.50 in API fees, making the system economically viable for a thesis-scale evaluation study.

Analytical query for per-call reporting. The following SQL query, run directly against the `llm_token_usage` table, provides a breakdown of observed token usage separated by call type:

```
SELECT
  call_name,
  COUNT(*)                          AS calls,
  ROUND(AVG(input_tokens))          AS avg_input,
  ROUND(AVG(output_tokens))         AS avg_output,
  SUM(input_tokens+output_tokens)AS total_tokens
FROM llm_token_usage
GROUP BY call_name
ORDER BY total_tokens DESC;
```

This enables direct comparison of *estimated* budgets (Table 5) against *observed* measurements from live sessions.

Known gaps and future work.

- **Thinking tokens not persisted.** Gemini 2.5 Flash allocates an internal reasoning budget (`thoughts_token_count`) that is billed separately. Capturing this field would give a complete cost picture.
- **Batch synthesis path.** The non-streaming `_synthesize_response()` function discards `usage_metadata`; this is only exercised in the batch `chat()` API and can be patched to log on that path as well.
- **Per-call API breakdown not exposed.** The analytics endpoint currently aggregates intent + synthesis together per session. A future version should expose per-`call_name` subtotals to allow cost attribution to individual pipeline stages.

6 Deployment Architecture

The production stack is orchestrated with Docker Compose across four services:

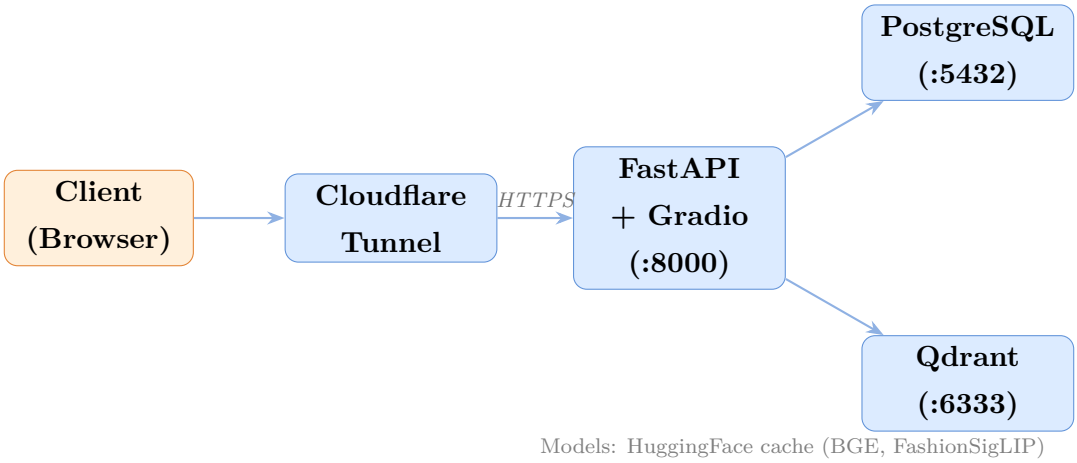


Figure 5: Production Docker Compose deployment. Cloudflare Tunnel provides a public HTTPS endpoint without exposing ports directly.

Service	Image	Port	Role
`postgres`	`postgres:16-alpine`	5432	Session memory + item store
`qdrant`	`qdrant/qdrant:latest`	6333	Vector similarity search
`fashion-api`	Custom build (Python 3.11)	8000	FastAPI + Gradio UI
`cloudflared`	`cloudflare/cloudflared`	—	Secure public tunnel

Table 6: Docker Compose service inventory.

7 Codebase Overview

Module	File	LOC
API Layer	`api/main.py`	376
Agent Orchestrator	`agent/fashion_agent.py`	442
Intent Classifier	`agent/intent_classifier.py`	126
Clarification Gate	`agent/clarification_gate.py`	110
Session Memory	`agent/memory.py`	288
Hybrid Search	`search/search_engine.py`	294
Query Expansion	`search/query_expansion.py`	106
RRF Fusion	`search/fusion.py`	72
BGE Reranker	`search/reranker.py`	115
Indexing Pipeline	`indexing/build_index.py`	470
Data Preprocessing	`pre_processing/processing_data.py`	698
Total		**3,097**

Table 7: Module inventory and lines-of-code breakdown.

8 Key Design Decisions

D1. 768-d FashionSigLIP over 512-d FashionCLIP. The higher-dimensional SigLIP representation captures richer fashion semantics and is fine-tuned on the target domain.

D2. Caption-as-semantic-token. Rather than embedding bare category labels, each item is indexed under a 3040-word LLM-generated description. This dramatically improves recall for descriptive style queries that would otherwise never match a short label.

D3. BM25 content limited to `label + color`. Including captions in the BM25 index degrades precision because long prose dilutes exact-match signals. Color and category label are the only fields where exact keyword matching adds value.

D4. RRF over score normalisation. BM25 and cosine similarity are on incompatible scales. Score normalisation introduces fragile assumptions; rank-based fusion is parameter-light and empirically robust.

D5. Cross-encoder reranker at post-fusion stage. Running BGE over all N candidates is too slow; running it only over the top-20 RRF results balances latency and

precision.

D6. PostgreSQL JSONB for session state. Storing query history and liked items as JSONB arrays avoids schema migrations as the session data model evolves. Atomic JSONB concatenation (`||`) is race-conditionsafe.

D7. Template-based slot gate (zero extra LLM calls). Pre-written clarification templates replace any iterative LLM loop. The gate releases after at most 3 clarification turns, bounding latency while preventing infinite loops.

9 Risks and Mitigations

Risk	Impact	Mitigation
High latency (multiple Gemini calls)	sequential LLM overhead eliminated by deterministic routing	Parallel tool calls; cache query expansion results
LLM hallucination	Agent fabricates product details	RAGAS faithfulness check; strict JSON output; fallback to product metadata only
BM25 rebuild on restart	Cold-start delay	Serialise BM25 index to disk; load at startup
PostgreSQL cold start	Connection spikes on container restart	Connection pooling (`psycopg2` pool); healthcheck service dependency
Gemini API rate limit	Batch jobs stall during indexing	Exponential backoff; asyncio throttle (20 items/batch)
Dataset label noise	Poor retrieval for edge categories	EXCLUDED_LABELS filter + LLM caption override

Table 8: Risk matrix with mitigations.

10 Conclusion and Future Work

Fashion Agent demonstrates that combining a *domain-specific multimodal embedding model* (Marqo-FashionSigLIP) with an *LLM orchestration layer* (Gemini 2.5 Flash via direct routing) yields a retrieval system that is simultaneously more precise (by leveraging structured intent and session memory) and more resilient (by clarifying ambiguity rather than guessing), while maintaining sub-15-second end-to-end latency.

Key contributions of Agent v2.0 over the RAG v1.0 baseline:

- Direct Routing Pipeline replacing a rigid 9-step pipeline.
- Intent classification eliminating irrelevant searches up front.
- Cross-encoder reranker (BGE v2-m3) improving precision without sacrificing recall.
- Persistent PostgreSQL session memory enabling coherent multi-turn dialogue.
- Bilingual response synthesis (English/Vietnamese).

Future directions.

Phase 3 (Scale) Expand to 15,000100,000 products; fine-tune the reranker on fashion-specific relevance labels; introduce Redis session caching to reduce PostgreSQL load.

Phase 4 (Advanced) Implement virtual try-on (IDM-VTON integration); fine-tune FashionSigLIP on proprietary inventory; A/B testing of reranker configurations.

PATH 2 (Image-to-Image) Allow users to upload a garment photo and retrieve visually similar items using composed image search.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

[2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023.

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, 2009.

[4] Z. Zhai et al., "Sigmoid Loss for Language Image Pre-Training," *ICCV*, 2023.

[5] S. Xiao et al., "C-Pack: Packaged Resources to Advance General Chinese Embedding," *arXiv:2309.07597*, 2023.

[6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," *SIGIR*, 2009.

[7] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv:2312.11805*, 2023.